



Angular component communication

SPRING 2026

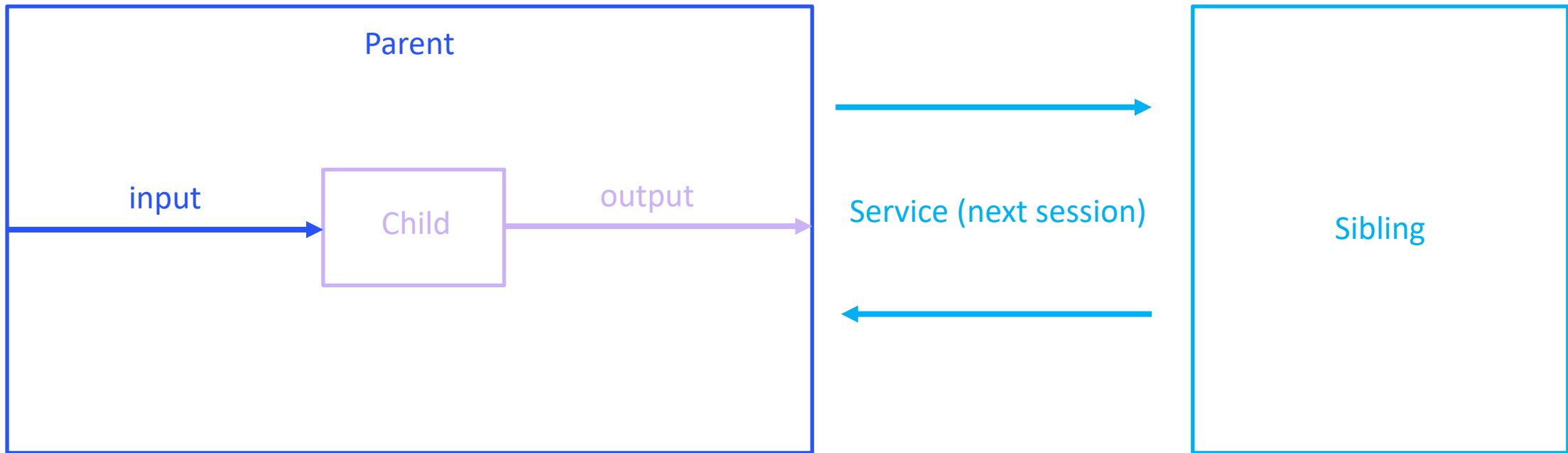
Overview

- Component communication
- Data binding
- Control flow
- Content projection
- Component lifecycle
- Signals
- Routing



Component communication

Basic data sharing



Parent to child: input

Parent component: passes data down to child

```
import { Component } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  template: `
    <h2>Parent Component</h2>

    <app-child [message]="'Hello from Parent 🙌'"></app-child>
  `
})
export class ParentComponent {}
```

Child component: receives input from parent

```
import { Component, input } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `
    <h3>Child Component</h3>
    <p>Message from parent: {{ message() }}</p>
  `
})
export class ChildComponent {
  // signal-based input
  message = input<string>();
}
```

Child to parent: output

Child component: emits event to parent

```
import { Component, output } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `
    <button (click)="sendToParent()">
      Click me (child)
    </button>
  `
})
export class ChildComponent {
  messageSent = output<string>();

  sendToParent() {
    this.messageSent.emit('Hi Parent! 🙌');
  }
}
```

Parent component: listens and reacts to event

```
import { Component } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  template: `
    <h2>Parent Component</h2>

    <app-child (messageSent)="showAlert($event)"></app-child>
  `
})
export class ParentComponent {
  showAlert(message: string) {
    alert(message);
  }
}
```

Direct access to child from parent using ViewChild

Parent accesses child method through child

```
import { Component, ViewChild } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  template: `
    <h2>Parent Component</h2>

    <app-child></app-child>

    <button (click)="callChild()">
      Call child method
    </button>
  `
})
export class ParentComponent {
  child = ViewChild(ChildComponent);

  callChild() {
    this.child()?.sayHello();
  }
}
```

Child method needed in parent

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `
    <p>I am the child component</p>
  `
})
export class ChildComponent {
  sayHello() {
    alert('Hello from Child 🙌');
  }
}
```

Data binding

The background of the slide features a series of fluid, overlapping waves in shades of deep purple, indigo, and a hint of blue. These waves create a sense of motion and depth, with some areas appearing brighter and more saturated than others, giving the overall composition a dynamic and modern feel.

Render dynamic text with text interpolation

You can bind dynamic text in templates with double curly braces, which tells Angular that it is responsible for the expression inside and ensuring it is updated correctly. This is called **text interpolation**.

```
@Component({
  template: `
    <p>Your color preference is {{ theme }}.</p>
  `,
  ...
})
export class App {
  theme = 'dark';
}
```

<!-- Rendered Output -->

```
<p>Your color preference is dark.</p>
```

The theme value here is constant and will not change.

If theme will change later, it should be a signal and will also update in the template accordingly (deeper dive into signals shortly).

Binding dynamic properties and attributes

In Angular, you use property bindings to set values directly to the DOM representation of the element.

```
<!-- Bind the `disabled` property on the button element's DOM object -->  
<button [disabled]="isFormValid()">Save</button>
```

In this example, every time `isFormValid` changes, Angular automatically sets the `disabled` property of the `HTMLButtonElement` instance.

Thanks to property binding and signals, anytime the signal changes, the `disabled` property will update and thus you will see the UI reflect the disabled state correctly.

Another property binding example: class

```
@Component({
  template: `
    <ul [class]="listClasses"> ... </ul>
    <section [class]="sectionClasses()"> ... </section>
    <button [class]="buttonClasses()"> ... </button>
  `,
  ...
})
export class UserProfile {
  listClasses = 'full-width outlined';
  sectionClasses = signal(['expandable', 'elevated']);
  buttonClasses = signal({
    highlighted: true,
    embiggened: false,
  });
}
```

Another property binding example: class

```
<ul class="full-width outlined"> ... </ul>
<section class="expandable elevated"> ... </section>
<button class="highlighted"> ... </button>
```

Endless property binding
possibilities...

Adding event listeners

Angular supports defining event listeners on an element in your template by specifying the event name inside parentheses along with a statement that runs every time the event occurs.

```
@Component({
  template: `
    <input type="text" (keyup)="updateField()" />
  `,
  ...
})
export class App{
  updateField(): void {
    console.log('Field is updated!');
  }
}
```

In this example, Angular calls `updateField` every time the `<input>` element emits a `keyup` event.

```
@Component({
  template: `
    <input type="text" (keyup)="updateField($event)" />
  `,
  ...
})
export class App {
  updateField(event: KeyboardEvent): void {
    console.log(`The user pressed: ${event.key}`);
  }
}
```

In every template event listener, Angular provides a variable named `$event` that contains a reference to the event object.

All sorts of events...

Another example: `<button (click)="doSomething()" />`

Two-way binding with form controls

Two-way binding is commonly used to keep component data in sync with a form control as a user interacts with the control. For example, when a user fills out a text input, it should update the state in the component.

```
import {Component} from '@angular/core';
import {FormsModule} from '@angular/forms';

@Component({
  imports: [FormsModule],
  template: `
    <main>
      <h2>Hello {{ firstName }}!</h2>
      <input type="text" [(ngModel)]="firstName" />
    </main>
  `,
})
export class App {
  firstName = 'Ada';
}
```

To use two-way binding with native form controls, you need to:

1. Import the FormsModule from @angular/forms
2. Use the ngModel directive with the two-way binding syntax (e.g., [(ngModel)])
3. Assign it the state that you want it to update (e.g., firstName)

Once that is set up, Angular will ensure that any updates in the text input will reflect correctly inside of the component state instantaneously!

Here, the name is initially Ada and is displayed as such in the template through string interpolation. As the user types inside the input, the firstName value gets updated instantaneously.

If not for this shortcut, you would've had to use event binding on input and assign the event's value to the firstName property to update it back in the template...

```
@Component({
  template: `
    <input [value]="username" (input)="onInput($event)">
    <p>Hello, {{ username }}</p>
  `,
})
export class ParentComponent {
  username = '';

  onInput(event: Event) {
    this.username = (event.target as HTMLInputElement).value;
  }
}
```

Basically, ngModel is just **property binding + event binding**, fused by naming convention.

Control flow

The background of the slide features a series of fluid, overlapping waves in shades of deep purple, indigo, and a hint of blue. These waves create a sense of motion and depth, with some areas appearing brighter and more saturated than others, giving the overall effect a dynamic and ethereal quality.

Conditionally display content

```
@if (a > b) {  
  <p>{{ a }} is greater than {{ b }}</p>  
}
```

```
@if (a > b) {  
  {{ a }} is greater than {{ b }}  
} @else if (b > a) {  
  {{ a }} is less than {{ b }}  
} @else {  
  {{ a }} is equal to {{ b }}  
}
```



Repeat content efficiently

The @for block loops through a collection and repeatedly renders the content of a block.

```
@for (item of items; track item.id) {  
  {{ item.name }}  
}
```

The track expression allows Angular to maintain a relationship between your data and the DOM nodes on the page. This allows Angular to optimize performance by executing the minimum necessary DOM operations when the data changes.

Select a property that uniquely identifies each item in the track expression. If your data model includes a uniquely identifying property, commonly id or uuid, use this value. Otherwise, you can use \$index to tell Angular to track each item by its index in the collection.

Content projection

The background of the slide features a series of fluid, overlapping waves in shades of deep purple, indigo, and a hint of magenta. These waves create a sense of motion and depth, with some areas appearing brighter and more saturated than others, giving the overall composition a dynamic and ethereal feel.

Dynamic component rendering

You may need to create components that act as containers for different types of content. For example, you may want to create a custom card component:

```
@Component({
  selector: 'custom-card',
  template: '<div class="card-shadow"> <!-- card content goes here --> </div>',
})
export class CustomCard {
  /* ... */
}
```

You can use the `<ng-content>` element as a placeholder to mark where content should go:

```
@Component({
  selector: 'custom-card',
  template: '<div class="card-shadow"> <ng-content/> </div>',
})
export class CustomCard {
  /* ... */
}
```

```
// Component source
@Component({
  selector: 'custom-card',
  template: `
    <div class="card-shadow">
      <ng-content />
    </div>
  `,
})
export class CustomCard {
  /* ... */
}
```

```
<!-- Using the component -->
<custom-card>
  <p>This is the projected content</p>
</custom-card>
```

Actual rendered dom

```
<div class="card-shadow">
  <p>This is the projected content</p>
</div>
```

- ng-content is replaced by the <p> element
- Could be any piece of code

=> dynamic

Dynamic multi content projection

```
<!--content-projector.component.html-->

<div>
  <h2>Header Content</h2>
  <ng-content select="[header]"></ng-content>
</div>
<div>
  <h2>Body Content</h2>
  <ng-content select="[body]"></ng-content>
</div>
<div>
  <h2>Footer Content</h2>
  <ng-content select="[footer]"></ng-content>
</div>
```

```
<!--app.component.html-->

<app-content-projector>
  <div header>
    <h3>This is the header content</h3>
  </div>
  <div body>
    <p>This is the body content</p>
  </div>
  <div footer>
    <p>This is the footer content</p>
  </div>
</app-content-projector>
```

Header Content

This is the header content

Body Content

This is the body content

Footer Content

This is the footer content



Component lifecycle

Lifecycle hooks

ngOnChanges

Called after a bound input property changes

ngOnInit

Called once the component is initialized

ngDoCheck

Called during every change detection run

ngAfterContentInit

Called after content (ng-content) has been projected into view

ngAfterContentChecked

Called every time the projected content has been checked

ngAfterViewInit

Called after the component's view (and child views) has been initialized

ngAfterViewChecked

Called every time the view (and child views) have been checked

ngOnDestroy

Called once the component is about to be destroyed

Signals

The background of the slide features a series of fluid, overlapping waves in shades of deep purple, indigo, and a hint of magenta. These waves create a sense of motion and depth, with some areas appearing brighter and more saturated than others, giving the overall effect a dynamic and ethereal quality.

A signal is a wrapper around a value that notifies interested consumers when that value changes.

Signals can contain any value, from primitives to complex data structures.

Signals may be either writable or read-only.



Writable signals

Writable signals provide an API for updating their values directly. You create writable signals by calling the `signal` function with the signal's initial value:

```
const count = signal(0);  
  
// Signals are getter functions - calling them reads their value.  
console.log('The count is: ' + count());
```

To change the value of a writable signal, either `.set()` it directly:

```
count.set(3);
```

or use the `.update()` operation to compute a new value from the previous one:

```
// Increment the count by 1.  
count.update((value) => value + 1);
```

Converting writable signals to readonly

This is useful when you want to expose a signal's value to consumers without allowing them to modify it directly

```
@Injectable({providedIn: 'root'})
export class CounterState {
  // Private writable state
  private _count = signal(0);

  readonly count = this._count.asReadonly(); // public readonly

  increment() {
    this._count.update((v) => v + 1);
  }
}
```

```
@Component({
  /* ... */
})
export class AwesomeCounter {
  state = inject(CounterState);

  count = this.state.count; // can read but not modify

  increment() {
    this.state.increment();
  }
}
```

The readonly signal reflects any changes made to the original writable signal, but cannot be modified using `set()` or `update()` methods.

Computed signals

Computed signals are read-only signals that derive their value from other signals. You define computed signals using the `computed` function and specifying a derivation:

```
const count: WritableSignal<number> = signal(0);  
const doubleCount: Signal<number> = computed(() => count() * 2);
```

The `doubleCount` signal depends on the `count` signal. Whenever `count` updates, Angular knows that `doubleCount` needs to update as well.

Computed signals are read-only.

```
doubleCount.set(3);
```



Computed signal dependencies are dynamic

Only the signals actually read during the derivation are tracked. For example, in this computed the count signal is only read if the showCount signal is true.

```
const showCount = signal(false);
const count = signal(0);
const conditionalCount = computed(() => {
  if (showCount()) {
    return `The count is ${count()}.`;
  } else {
    return 'Nothing to see here!';
  }
});
```

This means that if showCount is false and count's value changes, conditionalCount will not recompute until showCount becomes true.

Effects

An effect is code that runs automatically when a signal changes. It can be used to perform actions on signal change. Angular calls it for you when the data changes.

Example use: logging, calling API, etc.

Generally not for calculating values (use `computed()` for that)

```
import { Component, signal, effect } from '@angular/core';

@Component({
  selector: 'app-example',
  standalone: true,
  template: `
    <button (click)="count.update(c => c + 1)">
      Clicked {{ count() }} times
    </button>
  `
})
export class ExampleComponent {
  count = signal(0);

  constructor() {
    effect(() => {
      console.log('Count changed to:', count());
    });
  }
}
```

Untracked signals

Rarely, you may want to execute code which may read signals within a reactive function such as computed or effect without creating a dependency.

You can prevent a signal read from being tracked by calling its getter with `untracked`:

```
effect(() => {  
  console.log(`User set to ${currentUser()} and the counter is ${untracked(counter)}`);  
});
```

`untracked` is also useful when an effect needs to invoke some external code which shouldn't be treated as a dependency:

```
effect(() => {  
  const user = currentUser();  
  untracked(() => {  
    // If the `loggingService` reads signals, they won't be counted as  
    // dependencies of this effect.  
    this.loggingService.log(`User set to ${user}`);  
  });  
});
```

Linked signals

Simply put, a linked signal follows another signal, but can also be changed locally.

```
const shippingOptions = signal(['Ground', 'Air', 'Sea']);
const selectedOption = linkedSignal(() => shippingOptions()[0]);
console.log(selectedOption()); // 'Ground'

selectedOption.set(shippingOptions()[2]);
console.log(selectedOption()); // 'Sea'

shippingOptions.set(['Email', 'Will Call', 'Postal service']);
console.log(selectedOption()); // 'Email'
```

Without

```
import { signal, effect } from '@angular/core';

const parentValue = signal(10);
const localValue = signal(parentValue());

effect(() => {
  localValue.set(parentValue());
});
```

With

```
import { signal, linkedSignal } from '@angular/core';

const parentValue = signal(10);

const localValue = linkedSignal(() => parentValue());
```

Practical usecase example

```
import { Component, input, linkedSignal } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  template: `
    <input
      [value]="value()"
      (input)="onInput($event)"
    />
  `
})
export class ChildComponent {
  initialValue = input<string>();

  value = linkedSignal(() => this.initialValue());

  onInput(event: Event) {
    const input = event.target as HTMLInputElement;
    this.value.set(input.value);
  }
}
```

Routing

The background of the slide features a series of fluid, overlapping waves in shades of deep purple, indigo, and a hint of blue. These waves create a sense of motion and depth, with some areas appearing brighter and more saturated than others, giving the overall effect a dynamic and modern feel.

Routing pillars

Routing in Angular is comprised of three primary parts:

- 1.Routes** define which component displays when a user visits a specific URL.
- 2.Outlets** are placeholders in your templates that dynamically load and render components based on the active route.
- 3.Links** provide a way for users to navigate between different routes in your application without triggering a full page reload.

What's a route?

In Angular, a **route** is an object that defines which component should render for a specific URL path or pattern, as well as additional configuration options about what happens when a user navigates to that URL.

Most projects define routes in a separate file that contains routes in the filename.

```
import {Routes} from '@angular/router';
import {HomePage} from './home-page';
import {AdminPage} from './about-page';

export const routes: Routes = [
  {
    path: '',
    component: HomePage,
  },
  {
    path: 'admin',
    component: AdminPage,
  },
];
```

Tip: If you generated a project with Angular CLI, your routes are defined in `src/app/app.routes.ts`.

Route URL Paths

A route path is the URL Angular listens to in order to show a component.

If the url looks like <http://localhost/home>, the url will match to the home route and render always the same HomeComponent. Same goes for the about route.

```
const routes = [  
  { path: 'home', component: HomeComponent },  
  { path: 'about', component: AboutComponent }  
];
```

There also exists dynamic routes. Parts of the URL can change, but Angular still loads the same component.

```
const routes = [  
  { path: 'users/:id', component: UserDetailsComponent }  
];
```

/users/1
/users/42
/users/abc

All go to UserDetailsComponent.

: means “this part is dynamic”

id is just a variable name

Angular extracts it from the URL and can use it for specific component rendering (ex: fetch the correct user from a list using id in url and display that user’s details in page)

Special routes and route order

When you define routes, the order is important because Angular uses a first-match wins strategy. This means that once Angular matches a URL with a route path, it stops checking any further routes. As a result, always put more specific routes before less specific routes.

```
export const routes: Routes = [  
  // 1 Redirect empty path to /home  
  { path: '', redirectTo: 'home', pathMatch: 'full' },  
  
  // 2 Home page  
  { path: 'home', component: HomeComponent },  
  
  // 3 Static pages  
  { path: 'about', component: AboutComponent },  
  
  // 4 New user page → must come before dynamic ID  
  { path: 'users/new', component: UserCreateComponent },  
  
  // 5 Dynamic user details  
  { path: 'users/:id', component: UserDetailsComponent },  
  
  // 6 User list  
  { path: 'users', component: UserListComponent },  
  
  // 7 Wildcard → 404  
  { path: '**', component: NotFoundComponent }  
];
```

Default pathMatch strategy is “prefix”

For empty base path, use pathMatch full!

/users/new must come before /users/:id
Otherwise Angular treats "new" as an id and loads
UserDetailsComponent instead.

When you need to catch all routes for a specific path, the
solution is a wildcard route which is defined with the double
asterisk (**). Commonly used for the “Not Found page”

Router outlet

The RouterOutlet directive is a placeholder that marks the location where the router should render the component for the current URL.

```
import {Routes} from '@angular/router';
import {Home} from './home';
import {Products} from './products';

const routes: Routes = [
  {
    path: '',
    component: Home,
    title: 'Home Page',
  },
  {
    path: 'products',
    component: Products,
    title: 'Our Products',
  },
];
```

```
<app-header />
<!-- Angular inserts your route content here -->
<router-outlet />
<app-footer />
```

When a user visits /products, Angular renders the following:

```
<app-header />
<app-products />
<app-footer />
```

If the user goes back to the home page, then Angular renders:

```
<app-header />
<app-home />
<app-footer />
```

Routerlink

```
<!-- Navigates to /settings/notifications -->  
<a routerLink="notifications">Notifications</a>  
<a routerLink="/settings/notifications">Notifications</a>  
  
<!-- Navigates to /team/:teamId/user/:userId -->  
<a routerLink="/team/123/user/456">User 456</a>  
<a [routerLink]="['/team', teamId, 'user', userId]">Current User</a>
```

Angular routing allows you specify whether you want the path to be relative to the current URL or to the root domain based on whether the relative path is prefixed with a forward slash (/) or not.

Programmatic navigation

```
import { Component, inject } from '@angular/core';
import { Router, ActivatedRoute } from '@angular/router';

@Component({
  selector: 'app-nav-demo',
  standalone: true,
  template: `
    <h2>Programmatic Navigation Demo</h2>

    <button (click)="goHome()">1 Go Home (static absolute)</button>
    <button (click)="goAbout()">2 Go About (navigateByUrl)</button>
    <button (click)="viewUser(42)">3 View User 42 (relative)</button>

    <p>Current URL: {{ currentUrl }}</p>
  `
})
```

```
export class NavDemoComponent {
  router = inject(Router);
  route = inject(ActivatedRoute);

  get currentUrl() {
    return this.router.url;
  }

  // 1 Static absolute path
  goHome() {
    this.router.navigate(['/home']);
  }

  // 2 Full URL string
  goAbout() {
    this.router.navigateByUrl('/about');
  }

  // 3 Relative navigation
  viewUser(id: number) {
    // Assuming this component is rendered under /users
    this.router.navigate([id], { relativeTo: this.route });
  }
}
```

THANK YOU

Questions?

ADDRESS

Mkalles, main road, bld.
757, Lebanon

CONTACT

Phone +961 70 478 758
Email inmind.academy@inmind.ai
Website www.inmind.academy

SOCIAL MEDIA

